

RHEL Security Tools

Automated deployment, validation and teardown of enterprise security tooling across Red Hat Enterprise Linux estates — from a vSphere template to a verified host, with every step logged.

CrowdStrike Falcon 8.10.0-19402 · Tanium Client 7.8.4.1298 · EL7-EL10 · x86_64 + aarch64

Overview

This tooling installs and verifies five security components on RHEL-family servers. It grew out of a single installer script and now has four layers that share the same core logic: a host-side orchestrator, an SSH fan-out control plane, a container image, and a CI/CD pipeline.

Host orchestrator

`security.sh` runs on the target. Detects the platform, picks and validates packages, installs, reports.

Control plane

`deploy_targets.sh` fans out over SSH to many hosts with bounded concurrency and retries.

Container

AlmaLinux 9 image carrying the scripts and all 14 packages. Orchestrates outward; runs no agent itself.

Validation harness

`e2e_validate.sh` builds a VM from a template, installs, verifies, uninstalls, confirms.

The five components

COMPONENT	WHAT IT INSTALLS	VERIFIED BY
<code>crowdstrike</code>	Falcon sensor RPM, registers to the tenant CID	service active + AID assigned
<code>tanium</code>	Tanium client RPM + <code>tanium-init.dat</code>	service active
<code>cmdbsync</code>	Local account + validated sudoers policy	<code>visudo -c</code> passes
<code>sentinel</code>	Azure Arc <code>azcmagent</code> , connects to a resource group	agent reports Connected
<code>syslog</code>	rsyslog forwarding to two collectors	service active + config present

What it does

- Detects the platform from `/etc/os-release` and maps RHEL, CentOS, Rocky, AlmaLinux and Oracle Linux onto an `e1N` tag.
- Selects the right package out of 14 staged RPMs based on that tag and the architecture. No version is hardcoded.
- Validates before installing — header parse, name match, architecture fit and payload digest verification.
- Installs idempotently. Re-running reports `UNCHANGED` rather than reinstalling or bouncing a healthy service.
- Never blindly overwrites. Existing config is backed up to `<file>.sectools-bak.<timestamp>` first.
- Reports per host and per component so one failure never masks another.
- Redacts secrets from stdout and logs while keeping installation status fully visible.

Why digest validation matters. A staged Falcon RPM was 3.8 MB against a declared 155 MB payload — a truncated download that would have failed mid-install with a confusing error. `rpm -K` catches it before the host is touched.

Why the agents are not containerized

The container orchestrates; it never hosts an agent. Each of these is bound to its host, and a containerized copy would report the container instead of the VM:

AGENT	HOST BINDING
CrowdStrike Falcon	Kernel / eBPF sensor, host-unique agent ID
Tanium Client	Host systemd, hardware and patch inventory
Azure Arc	Registers the machine identity in Azure
rsyslog	Owns the host logging pipeline
cmdbsync	Local user, sudoers, host discovery commands

Tech stack and where each piece is used

TECHNOLOGY	USED FOR	WHERE
Bash (<code>strict mode</code>)	All orchestration. <code>set -Eeuo pipefail</code> , traps, explicit exit codes	<code>security.sh</code> , <code>lib/</code> , <code>scripts/*.sh</code>
Python 3 + pyVmomi 8.0.2	vSphere API: locate template, clone, NIC, power on, await IP	<code>scripts/vsphere_provision.py</code>
rpm	Header inspection and <code>rpm -K</code> payload digest verification	<code>lib/pkg_select.sh</code> , container build, CI
OpenSSH	Transport to every target; secrets on a separate channel	<code>deploy_targets.sh</code> , <code>e2e_validate.sh</code>
sshpas	Password auth for freshly cloned templates without keys	<code>e2e_validate.sh</code> only
tar + gzip	88 MB per-host bundle streamed over SSH	<code>stage_bundle.sh</code>
systemd	Service enable, restart and <code>is-active</code> verification	<code>security.sh</code> on targets
Podman / Docker	Control-plane image, AlmaLinux 9 base, non-root	<code>Dockerfile</code> , <code>docker-compose.yml</code>
AlmaLinux 9	Base image — native <code>rpm</code> , deliberately independent of target OS	<code>Dockerfile</code>
GitHub Actions	Lint, manifest, secret hygiene, platform matrix; gated deploys	<code>.github/workflows/</code>
subscription-manager	RHSM register / unregister, repos per EL version	<code>sub-reg.sh</code>
Azure Arc (<code>azcmagent</code>)	Sentinel onboarding via service principal	<code>sentinel_core.sh</code>

System diagram

Two entry points share the same target-side logic. The control plane orchestrates over SSH; it never runs a security agent itself.

Path A — Template validation (vSphere)





Path B — Fleet deploy (existing servers)



CONTROL PLANE

Jump VM or `sectools-deployer` container · AlmaLinux 9 · Bash

target.sh

Catalogue of FW/BLR templates; wraps e2e and fleet commands.

Tech: Bash · OpenSSH client

e2e_validate.sh

Clone or convert template → place scripts → install → verify → uninstall → finalize.

Tech: Bash · sshpass · Python 3

vsphere_provision.py

Find template, clone/convert VM, attach NIC, power on, wait for IP. Scope locked to SnowCommander/Templates.

Tech: Python 3 · pyVmomi · vCenter API :443

stage_bundle.sh

Read `deploy-manifest.txt`; pick 1 Falcon + 1 Tanium for probed EL/arch; validate digests; emit ~88 MB tar.gz.

Tech: Bash · rpm · tar · gzip · lib/pkg_select.sh

deploy_targets.sh

Probe each host, stage bundle, push over SSH, run `security.sh`, collect status. Bounded parallelism.

Tech: Bash · OpenSSH · GNU parallel logic

entrypoint.sh

Container commands: `plan`, `verify`, `deploy`, `packages`.

Tech: Podman/Docker · read-only rootfs · secrets mounted at runtime

Ships: 14 RPMs + all scripts · Never ships: live secrets · Never runs: agents

SSH :22

bundle || secrets

TARGET HOST

RHEL / CentOS / Rocky / Alma / Oracle · EL7–EL10 · x86_64 or aarch64

/home/tpx-admin/2026snowcommander/
Scripts extracted here. No sudo needed for the file transfer.
Transport: scp / tar over SSH

securitytools.env
Secrets on a separate SSH channel into mode 0600 ; shredded after the run.
Tech: Bash · OpenSSH

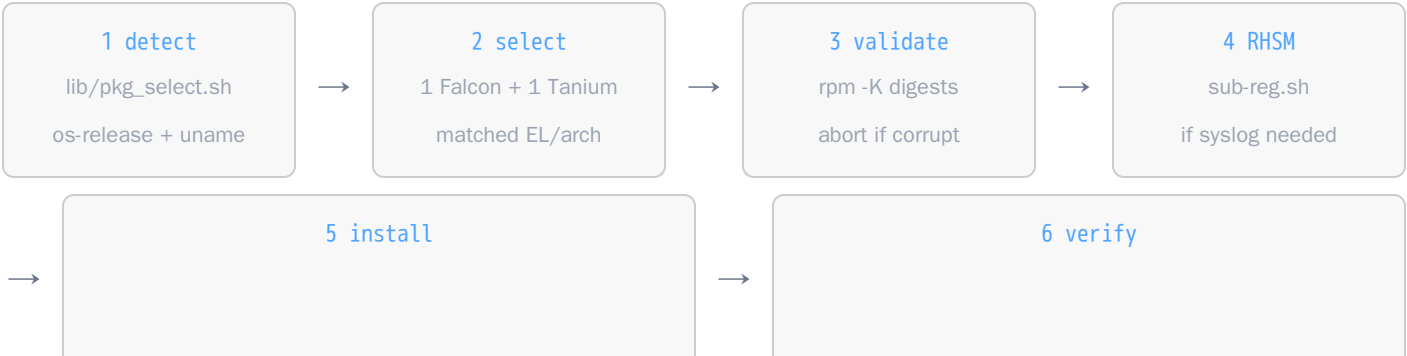
sub-reg.sh
Register with RHSM before syslog install; unregister appliance templates after verify.
Tech: subscription-manager · yum/dnf

security.sh
Main orchestrator — detect platform, select packages, install five components, report status.
Tech: Bash · sudo · systemd

lib/pkg_select.sh
/etc/os-release → eIN tag · uname -m → arch · rpm -K digest check.
Tech: Bash · rpm

Installed services
CrowdStrike falcon-sensor · Tanium taniumclient · rsyslog · Azure Arc azcmagent · local cmdbsync user.
Tech: systemd · kernel sensor · Azure SP auth

On the target — install sequence



security.sh all

5 components

systemctl + status block

per component

Five components installed by security.sh

crowdstrike

Falcon RPM + **falconctl** CID

kernel sensor

systemd

tanium

TaniumClient RPM + init dat

systemd

glibc build

cmdbsync

Local user + sudoers

visudo

sentinel

sentinel_core.sh → azcmagent

Azure SP

Arc

syslog

rsyslog + remote conf

yum/dnf

RHSM

End-to-end flow

The validation harness runs eight stages and prints a pass/fail matrix at the end.

- 1** Connect to vSphere
 pyVmomi **SmartConnect** to vCenter on 443.
- 2** Find the template
 Search restricted to **SnowCommander/Templates**. Anything outside is invisible and refused with exit 7.
- 3** Template → VM
 Clone (template preserved), connect the NIC at both **connected** and **startConnected**, power on, wait for an IPv4 address.
- 4** Place the scripts
 Remove the stale **2026snowcommander** payload, build a matched 88 MB bundle, extract to **/home/tpx-admin/2026snowcommander**, and add the FQDN to **/etc/hosts**. Secrets follow on a separate channel into a 0600 file.
- 5** Install the tools
 RHSM register if needed, then **security.sh all** — detect, select, validate, install, configure.
- 6** Verify with systemctl
systemctl is-active per service, **rpm -q** per package, plus structured verification.

7

Uninstall the tools

Remove the three agents — Tanium, CrowdStrike and Sentinel. The `cmdbsync` account and the Red Hat subscription are deliberately retained, and stage 7 re-checks both to prove they survived. Appliance templates unregister from RHSM here.

8

Confirm the scripts remain

Assert every expected script is still present, then shred `securitytools.env`.

With `--template-cycle` a ninth step follows: shut the guest down cleanly and call `MarkAsTemplate`, so the validated machine becomes a golden image again.

Failure is self-healing. If the run ends before finalisation, an `EXIT` trap converts the object back to a template anyway. A crash between stage 3 and the end cannot leave your golden image stranded as a VM.

What the end state looks like

ITEM	STATE AFTER A FULL CYCLE
<code>/home/tpx-admin/2026snowcommander</code>	Current scripts, ready to re-run
Stale <code>2026snowcommander</code> payload	Removed, including the superseded RPMs
<code>securitytools.env</code>	Shredded — no credentials left behind
Tanium, CrowdStrike, Sentinel	Uninstalled
<code>cmdbsync</code> account and sudoers	Retained
Red Hat subscription	Retained for <code>*Redhat*</code> templates
The vSphere object	A template again

Mechanism

Platform detection

Reads `ID` and `VERSION_ID` from `/etc/os-release`, maps the family to an `eLN` tag, and normalises the architecture from `uname -m`. Debian, Ubuntu and Fedora are rejected with exit 3 and an explicit message rather than being silently mis-served — no `.deb` packages exist here.

Names lie, metadata doesn't. The template `BLR-GI-6.6` is Vocera 6.6 running on RHEL 7.9, not RHEL 6.6. Detection reads the OS, never the label. vSphere may report an older guest OS string; trust `/etc/os-release` and the structured `arch=` line in the status block.

Idempotency

Every component compares desired state against actual state:

- CrowdStrike — skips entirely when build, CID and service state all match. CID compared as bare 32 hex characters, since `falconctl` reports it lowercase and without the checksum suffix.
- Tanium — restarts only if the RPM or `tanium-init.dat` actually changed.
- cmdbsync — never resets an existing account's password unless `CMDBSYNC_FORCE_PASSWORD=1`.
- Sentinel — skipped when already Connected; Disconnected agents are force-disconnected locally then reconnected. Expired `AZCM_SP_SECRET` fails with an actionable error.
- syslog — writes only on content change, validates with `rsyslogd -N1`, rolls back on failure.

Secret handling

Secrets are never baked into an image, written to a tarball, or passed as command arguments where `ps` could read them. On the jump host they live in `~/snowcommander-creds.env.vault` (ansible-vault encrypted); `target.sh` decrypts to a temp file in memory for the current process only. On targets they travel on a dedicated SSH channel into a 0600 `securitytools.env` and can be shredded after install. A `sed` filter masks secret *values* in logs — including output from `dnf`, `falconctl` and `azcmagent`. The CID is deliberately not redacted: it is an identifier, not a credential.

Scripts — what each one does

Core

SCRIPT	LINES	RUNS ON	PURPOSE
<code>security.sh</code>	1071	target	Main orchestrator. Modes: <code>all</code> , individual components, <code>verify</code> , <code>plan</code> .
<code>lib/pkg_select.sh</code>	459	both	OS→eIN detection, package selection, five-point validation incl. digests.
<code>uninstall.sh</code>	1240	target	Removes packages, services, directories, users and sudoers. Arc uninstall is time-boxed and skips hung <code>systemctl stop</code> after disconnect.
<code>sub-reg.sh</code>	185	target	RHSM <code>register</code> / <code>unregister</code> / <code>status</code> , repos per EL version.
<code>sentinel_core.sh</code>	260	target	Azure Arc onboarding via service principal; force-local disconnect, unique resource names, credential validation.

Orchestration

SCRIPT	LINES	PURPOSE
<code>scripts/e2e_validate.sh</code>	1070	The 8-stage harness. Per-stage logging, guest SSH remedy, final matrix.
<code>scripts/vsphere_provision.py</code>	870	vSphere stages. Scope-locked to <code>SnowCommander/Templates</code> ; lists VMs mid template-cycle.
<code>scripts/creds-vault.sh</code>	130	Encrypt/decrypt <code>~/snowcommander-creds.env</code> with ansible-vault.
<code>scripts/deploy_targets.sh</code>	308	SSH fan-out with bounded concurrency and retries.
<code>scripts/stage_bundle.sh</code>	189	Builds the per-target bundle from the manifest.
<code>target.sh</code>	430	Single entry point: FW/BLR/STC catalogue, <code>e2e-stc</code> , <code>e2e-site</code> , fleet deploy; auto-decrypts vault creds.
<code>entrypoint.sh</code>	155	Container entry: <code>plan</code> , <code>verify</code> , <code>deploy</code> , <code>packages</code> , <code>bundle</code> .

Standalone component installers

Shipped to targets for manual incident-response use. `security.sh` does not call them — it has its own inlined component logic.

`install_crowdstrike.sh` · `install_tanium.sh` · `install_cmdbsync.sh` · `install_sentinel.sh` · `install_syslog.sh`

CPU architecture

Every target is classified as `x86_64` or `aarch64`. The control plane probes `uname -m` over SSH (or reads it on the host when `security.sh` runs locally), then ships exactly one CrowdStrike sensor and one Tanium client matched to that architecture. A mixed inventory — some `x86_64` hosts and some `aarch64` — is fine in a single deploy run.

UNAME -M	NORMALISED	NOTES
<code>x86_64</code> , <code>amd64</code>	<code>x86_64</code>	Default for FW/BLR SnowCommander templates today
<code>aarch64</code> , <code>arm64</code>	<code>aarch64</code>	Supported from EL8 upward for Falcon; Tanium uses a separate glibc build
anything else	rejected	Exit code <code>4</code> — no staged package exists

Staged package matrix (14 RPMs)

CrowdStrike ships genuinely distinct per-EL builds. Tanium ships two payloads keyed by architecture, not by the `rheN` in the filename.

EL	X86_64	AARCH64
el7	Falcon ✓ · Tanium ✓	Falcon not published · Tanium ✓
el8	Falcon ✓ · Tanium ✓	Falcon ✓ · Tanium ✓
el9	Falcon ✓ · Tanium ✓	Falcon ✓ · Tanium ✓
el10	Falcon ✓ · Tanium ✓	Falcon ✓ · Tanium ✓

Tanium is arch-keyed, not version-keyed. Seven filenames, two unique payloads: `glibc2.17.systemd` for `x86_64` and `glibc2.26.systemd` for `aarch64`. Selection reads RPM header metadata when the cosmetic `rheN` filename does not match.

el7 aarch64 + CrowdStrike. No el7 aarch64 Falcon sensor is staged. Deploys to such a host fail at bundle time with a clear message — they are not silently given an `x86_64` package.

Package selection

14 RPMs staged; exactly two are shipped per host. Selection is by glob with the highest version winning, so staging a newer sensor needs no code change. See [CPU architecture](#) for the full EL × arch matrix.

TARGET	CROWDSTRIKE	TANIUM
el7 x86_64	✓	✓
el7 aarch64	not published	✓ via arch match
el8 / el9 / el10 x86_64	✓	✓
el8 / el9 / el10 aarch64	✓	✓

Validation gate

Every candidate must pass all five checks or the run aborts before the host is modified: file present and non-empty, header parseable, package name matches, architecture installable, payload digests verify.

vSphere templates — FW, BLR & STC

FW and BLR each have six templates under `SnowCommander/Template(s)`. STC is scoped to a single RHEL target: `SC-Redhat 9` (quote the name — it contains a space). All current golden images are x86_64. The harness detects the real OS and architecture at runtime — never from the template name.

Configured in `target.sh`. List them with `./target.sh list`; validate every template on one site with `./target.sh e2e-site fw --template-cycle`.

Fort Wayne (FW)

TEMPLATE	ARCHITECTURE	TYPE	TYPICAL GUEST OS	RHSM AFTER CYCLE
FW-Redhat-9	x86_64	Red Hat base	RHEL 9	keep
FW-Redhat-10	x86_64	Red Hat base	RHEL 10	keep
FW-GI-6.6	x86_64	Vocera / GI appliance	RHEL 7.9 (name ≠ OS version)	remove
FW-GI-7.1	x86_64	Vocera / GI appliance	RHEL 7.x	remove
FW-GI-7.2	x86_64	Vocera / GI appliance	RHEL 7.x	remove
FW-GI-7.3	x86_64	Vocera / GI appliance	RHEL 7.x	remove

vCenter: fw-vsphere-01.strykercorp.com · portgroup: fwa-vlan106 · FQDN domain: vcraeng.com

Bangalore (BLR)

TEMPLATE	ARCHITECTURE	TYPE	TYPICAL GUEST OS	RHSM AFTER CYCLE
BLR-Redhat-9	x86_64	Red Hat base	RHEL 9	keep
BLR-Redhat-10	x86_64	Red Hat base	RHEL 10	keep
BLR-GI-6.6	x86_64	Vocera / GI appliance	RHEL 7.9 (name $\neq$ OS version)	remove
BLR-GI-7.1	x86_64	Vocera / GI appliance	RHEL 7.x	remove
BLR-GI-7.2	x86_64	Vocera / GI appliance	RHEL 7.x	remove
BLR-GI-7.3	x86_64	Vocera / GI appliance	RHEL 7.x	remove

vCenter: blr-vsphere-01.strykercorp.com · portgroup: VLAN-126 · FQDN: vcraeng.com

St Cloud (STC)

TEMPLATE	ARCHITECTURE	TYPE	TYPICAL GUEST OS	RHSM AFTER CYCLE
SC-Redhat 9	x86_64	Red Hat base	RHEL 9	keep

vCenter: stc-vsphere-01.strykercorp.com · portgroup: stc-vlan847 · FQDN: none

Folder: **IT/SnowComander/Templates** . Run **./target.sh list-vcenter stc** or **./target.sh e2e-stc --template-cycle** .

Package picked per detected EL + arch. A GI template on RHEL 7.9 x86_64 receives **falcon-sensor-*
el7.x86_64.rpm** and the glibc2.17 Tanium x86_64 build, regardless of the **GI-6.6** label.

Credentials — where they are set

Four sets, one file, kept outside the repository. Everything is read from the environment; nothing is ever a command-line argument.

```
# Create it once
cd /home/tpx-admin/snowcommander_scripts
cp credentials.env.example ~/.snowcommander-creds.env
chmod 600 ~/.snowcommander-creds.env
vi ~/.snowcommander-creds.env

# Encrypt (recommended – see #creds-encrypt)
./scripts/creds-vault.sh init && ./scripts/creds-vault.sh encrypt
```

```
# Or load plain file temporarily (before encrypt)
set -a; source ~/.snowcommander-creds.env; set +a
```

SET	VARIABLES	USED BY
vSphere	<code>VCENTER_USER</code> , <code>VCENTER_PASSWORD</code>	Stages 1–3
Target VM	<code>TARGET_SSH_USER</code> + <code>TARGET_SSH_KEY</code> or <code>SSHPASS</code>	Stages 4–8
Security tools	<code>FALCON_CID</code> , <code>CMDBSYNC_PASSWORD</code> , <code>AZCM_*</code>	Stage 5
Red Hat	<code>RHSM_USERNAME</code> , <code>RHSM_PASSWORD</code>	Stage 5 register/unregister

No provisioning token. This tenant does not require one. It is absent from every template and rendering path; `security.sh` still passes `--provisioning-token` if the variable is ever set, so a tenant that needs one works without a code change.

Least privilege. Give the vCenter account Clone and Power On on `SnowCommander/Templates` only, so the restriction is enforced by vCenter permissions as well as by the code.

Encrypt credentials at rest

After filling in plain `~/.snowcommander-creds.env`, encrypt it with `ansible-vault`. You never type a password at runtime — a random password file is generated once and reused automatically.

```
cd /home/tpx-admin/snowcommander_scripts

# 1. Create vault password file (once)
./scripts/creds-vault.sh init
# -> ~/.snowcommander-vault.pass (mode 600) – back this up securely

# 2. Encrypt plain creds
./scripts/creds-vault.sh encrypt
# -> ~/.snowcommander-creds.env.vault
# -> shreds plain .env; leaves a timestamped .plain.bak backup

# 3. Confirm
./scripts/creds-vault.sh status
```

FILE	PURPOSE
<code>~/snowcommander-creds.env.vault</code>	Encrypted credentials (ansible-vault)
<code>~/snowcommander-vault.pass</code>	Vault password — required to decrypt; never commit
<code>SNOWCOMMANDER_VAULT_PASS_FILE</code>	Optional — point at an existing ansible vault password file

Runtime: `./target.sh`, `./target.sh e2e-stc`, and `./target.sh container deploy` call `load_creds`, which decrypts the vault to a temp file in memory — no prompt, no plain file left on disk.

For direct `e2e_validate.sh` (without `target.sh`), load creds manually:

```
set -a; source <(/scripts/creds-vault.sh materialize); set +a
```

Other commands: `edit`, `view`, `decrypt` — see `./scripts/creds-vault.sh help`.

Red Hat subscriptions

rsyslog installs from Red Hat repositories, so an unregistered RHEL host cannot complete the syslog component. `--rhsm-mode auto` (the default) decides from the template name:

TEMPLATE	BEHAVIOUR
<code>*Redhat*</code>	Register, install, verify — stays registered
<code>*GI*</code> , <code>*Engage*</code>	Register, install, verify — unregisters after stage 6
anything else	Skipped; override with <code>--rhsm-mode keep remove</code>

Repository names are derived per version, not hardcoded:

DETECTED	REPOSITORIES ENABLED
el7	<code>rhel-7-server-rpms</code> , <code>rhel-7-server-extras-rpms</code>
el8 / el9 / el10	<code>rhel-N-for-<arch>-baseos-rpms</code> , <code>---appstream-rpms</code>

One-time setup

Do this once on the jump VM before validating templates or running fleet deploys.

```
cd /home/tpx-admin/snowcommander_scripts

# 1. Credentials (never commit this file)
cp credentials.env.example ~/.snowcommander-creds.env
chmod 600 ~/.snowcommander-creds.env
# Edit: VCENTER_USER, VCENTER_PASSWORD, SSHPASS, FALCON_CID, AZCM_* ...
./scripts/creds-vault.sh init
./scripts/creds-vault.sh encrypt

# 2. Stage vendor RPMs (~548 MB – not in git)
cp -r /opt/snowcommander-packages/crowdstrike ./crowdstrike
cp -r /opt/snowcommander-packages/tanium ./tanium
cp /opt/snowcommander-packages/tanium/tanium-init.dat ./tanium/

# 3. Build the deployer container (needed for fleet deploy, not for e2e)
podman build -t sectools-deployer:latest .

# 4. Confirm the catalogue
./target.sh list
```

Execution methodology

Three phases, two runtimes. Never mix template validation (jump host + vCenter) with fleet deploy (container + inventory).

1

Prepare jump host

Clone repo, stage RPMs, encrypt creds, build container image

2

Validate templates (e2e)

Jump host only — `target.sh e2e*` talks to vCenter, SSHs to guest, install → verify → uninstall → restore template

3

Deploy to fleet

Container or jump host — SSH to live hostnames in `inventory`, no vCenter

Phase 2 — template validation flow (8 stages)

STAGE	WHAT HAPPENS
1–3	Connect vCenter, find template (or reuse VM mid-cycle), power on, get IP, attach NIC to site portgroup
4	SCP scripts to <code>/home/tpx-admin/2026snowcommander</code> on guest
5	<code>security.sh all</code> — Tanium, CrowdStrike, cmdbsync, Sentinel, syslog
6	Verify services and packages
7	<code>uninstall.sh tanium crowdstrike sentinel</code> — retains cmdbsync + RHSM subscription on Red Hat templates
8	Confirm scripts remain; optionally keep or shred <code>securitytools.env</code>
9	(template-cycle only) Power off and convert VM back to template

Site quick reference

SITE	ONE-LINER	NOTES
FW	<code>./target.sh e2e fw FW-GI-7.3 --template-cycle --insecure</code>	6 templates; FQDN <code>vcraeng.com</code>
BLR	<code>./target.sh e2e blr BLR-Redhat-9 --template-cycle --insecure</code>	6 templates; portgroup <code>VLAN-126</code>
STC	<code>./target.sh e2e-stc --template-cycle --insecure</code>	<code>SC-Redhat 9</code> only; portgroup <code>stc-vlan847</code> ; quote names with spaces

Other STC templates (e.g. GI) — confirm the object lives in `IT/SnowComander/Templates`, then:

```
./target.sh e2e stc 'SC-GI 7.1' --template-cycle --insecure
```

GI names vary by site; list inventory first:

```
set -a; source <(/scripts/creds-vault.sh materialize); set +a
python3 scripts/vsphere_provision.py \
  --vcenter stc-vsphere-01.strykercorp.com --list-templates --insecure
```

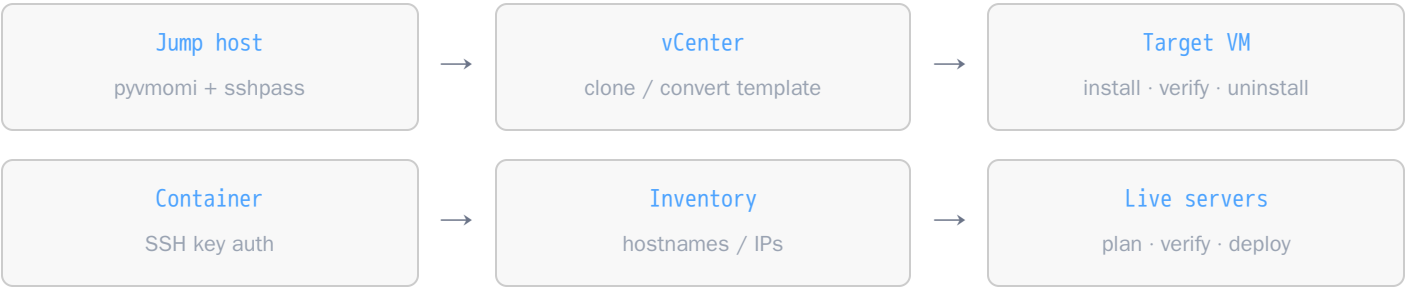
Creds: `target.sh` auto-decrypts the vault — no manual `source` step. Only direct `e2e_validate.sh` calls need `source <(/scripts/creds-vault.sh materialize)`.

Jump host vs container — which path to use

These are two different workflows. Do not mix them up.

GOAL	COMMAND	RUNS ON
Validate STC SC-Redhat 9	<code>./target.sh e2e-stc --template-cycle --insecure</code>	Jump host
Validate one template on vSphere	<code>./target.sh e2e fw FW-GI-7.3 --template-cycle</code>	Jump host
Validate all templates on one vCenter	<code>./target.sh e2e-site fw --template-cycle</code>	Jump host
Validate all three vCenters	<code>./target.sh e2e-all-sites --template-cycle</code>	Jump host
Deploy tools to live servers	<code>./target.sh container deploy</code>	Container

Template validation does not use the container. The e2e harness needs `pyvmomi` (vSphere API) and `sshpass` (password auth on fresh templates) on the jump VM. The `sectools-deployer` container is for production fleet deploys over SSH keys to hostnames in an inventory file — it never talks to vCenter.



Smoke checks before a full run

```
# Prove vCenter credentials and list templates in scope (read-only)
./target.sh list-vcenter fw

# Dry run against a host that already exists - no VM is created
./scripts/e2e_validate.sh --existing-host blr-gi-6 --keep-tools
```

vCenter reference

`target.sh` knows three sites. Portgroups and FQDN defaults are baked in; override with `--portgroup` or environment variables (`BLR_PORTGROUP`, `FW_FQDN_DOMAIN`, ...).

SITE	VCENTER	DEFAULT PORTGROUP	FQDN	TEMPLATES
fw	fw-vsphere-01.strykercorp.com	fwa-vlan106	vcraeng.com	FW-Redhat-9, FW-Redhat-10, FW-GI-6.6, FW-GI-7.1, FW-GI-7.2, FW-GI-7.3
blr	blr-vsphere-01.strykercorp.com	VLAN-126	vcraeng.com	BLR-Redhat-9, BLR-Redhat-10, BLR-GI-6.6, BLR-GI-7.1, BLR-GI-7.2, BLR-GI-7.3
stc	stc-vsphere-01.strykercorp.com	stc-vlan847	—	SC-Redhat 9 (quote names with spaces)

Recommended order per vCenter

1. `./target.sh list-vcenter fw` — confirm templates exist in vCenter
2. `./target.sh e2e fw FW-GI-6.6 --template-cycle` — one GI template end-to-end first
3. `./target.sh e2e-site fw --template-cycle` — all six templates on that vCenter
4. Repeat steps 1–3 for `blr`, then `stc`
5. After VMs are provisioned from validated templates → `./target.sh container deploy`

One template at a time

Runs on the jump host, not in a container. Use `./target.sh` — it decrypts encrypted creds automatically. For bare `e2e_validate.sh`, run `source <(.scripts/creds-vault.sh materialize)` first.

Two ways to validate a template. Pick based on whether you want a second object in the inventory afterwards.

Round trip on the template itself (recommended)

No new VM. The template becomes a VM, is validated, and is converted back. `target.sh` supplies the correct vCenter, portgroup and FQDN per site.

FW — one template

```
./target.sh e2e fw FW-GI-7.3 --template-cycle
```

BLR — one template

```
./target.sh e2e blr BLR-GI-6.6 --template-cycle
```

STC — SC-Redhat 9

```
./target.sh e2e-stc --template-cycle
```

Shortcut for the only STC RHEL template. Equivalent to `./target.sh e2e stc 'SC-Redhat 9' --template-cycle`.

Explicit form (same thing, all flags visible)

```
./scripts/e2e_validate.sh \
  --vcenter fw-vsphere-01.strykercorp.com \
  --template FW-GI-7.3 \
  --template-cycle \
  --portgroup 'fwa-vlan106' \
  --fqdn-domain vcraeng.com \
  --insecure

./scripts/e2e_validate.sh \
  --vcenter stc-vsphere-01.strykercorp.com \
  --template 'SC-Redhat 9' \
  --template-cycle \
  --portgroup 'stc-vlan847' \
  --insecure
```

`--template-cycle` is shorthand for `--mode convert`, the destructive acknowledgement, and `--convert-to-template`.

Clone into a new VM

Leaves the template untouched and produces a second object.

```
./scripts/e2e_validate.sh \
  --vcenter fw-vsphere-01.strykercorp.com \
  --template FW-Redhat-9 \
  --name e2e-fw-rhel9 \
  --portgroup 'fwa-vlan106' \
  --insecure
```

Useful variations

```
# keep the tools installed for inspection (skip stage 7)
./scripts/e2e_validate.sh ... --keep-tools

# force subscription behaviour instead of inferring it from the name
./scripts/e2e_validate.sh ... --rhsm-mode remove

# install only some components
./scripts/e2e_validate.sh ... --components "crowdstrike tanium"

# change what stage 7 tears down (default: the three agents)
./scripts/e2e_validate.sh ... --uninstall-components "tanium crowdstrike"

# different script location on the target
./scripts/e2e_validate.sh ... --remote-dir /opt/snowcommander

# validate a host that already exists, skipping stages 1-3
./scripts/e2e_validate.sh --existing-host blr-gi-6 --keep-tools
```

No guest customization is applied. The VM boots with the template's own hostname and static IP. Two clones of the same template must never run at once, and the address must be free before you start.

Every template in one vSphere

Runs sequentially on the jump host. Each template is round-tripped; logs go to `/tmp/e2e-<site>-<template>.log`.

```
# All six FW templates
./target.sh e2e-site fw --template-cycle --insecure

# All six BLR templates
./target.sh e2e-site blr --template-cycle --insecure

# STC – SC-Redhat 9 only
./target.sh e2e-stc --template-cycle --insecure

# Watch a run in progress
tail -f /tmp/e2e-fw-FW-GI-7.3.log
```

All three vSphere at once

FW, BLR and STC run in parallel background jobs — one log file per template under `/tmp/e2e-runs/`. A combined summary is printed when all sites finish.

```
./target.sh e2e-all-sites --template-cycle --insecure
```

```
# Review results
```

```
column -t /tmp/e2e-runs/summary.txt
```

```
tail /tmp/e2e-runs/fw-FW-GI-7.3.log
```

Run one template end to end first. Parallel runs across three sites convert templates simultaneously. Confirm a single clean pass on each vCenter before scaling out.

Container fleet deploy — existing servers

Use this after templates are validated and VMs are running in production. The container SSHs to hostnames in an inventory file — it does not talk to vCenter and does not accept template names.

Prepare inventory

```
./target.sh write-inventory ./inventory
```

```
vi ./inventory # one hostname or IP per line; comments with #
```

Option 1 — `target.sh container` (simplest)

```
# Decrypts ~/.snowcommander-creds.env.vault automatically
```

```
./target.sh container plan
```

```
./target.sh container verify
```

```
./target.sh container deploy
```

Option 2 — `podman run` directly

Requires a plain env file — decrypt first: `./scripts/creds-vault.sh materialize > /tmp/creds.env && chmod 600 /tmp/creds.env`

```
podman run --rm \
  --env-file /tmp/creds.env \
  -v ~/.ssh/id_ed25519:/run/secrets/ssh_key:ro \
```

```
-v ./inventory:/etc/snowcommander/inventory:ro \
-e INVENTORY=/etc/snowcommander/inventory \
-e PARALLEL=5 \
sectools-deployer:latest plan

# then verify, then:
podman run --rm \
  --env-file /tmp/creds.env \
  -v ~/.ssh/id_ed25519:/run/secrets/ssh_key:ro \
  -v ./inventory:/etc/snowcommander/inventory:ro \
  -e INVENTORY=/etc/snowcommander/inventory \
  sectools-deployer:latest deploy all
rm -f /tmp/creds.env
```

Option 3 — docker compose

```
export INVENTORY_PATH=./inventory
export SSH_KEY_PATH=~/.ssh/id_ed25519
docker compose run --rm plan
docker compose run --rm verify
docker compose run --rm deploy
```

Option 4 — jump host without container

Same logic as the container, runs directly on the VM (needs SSH key in environment):

```
./target.sh plan
./target.sh verify
./target.sh deploy
```

The container orchestrates; it never hosts an agent. CrowdStrike, Tanium and Azure Arc must run on the target OS. The container only opens outbound SSH, pushes the bundle, and runs `security.sh` remotely.

GitHub pipeline

Two workflows in `.github/workflows/`.

CI — automatic on every push

JOB	ENFORCES
shell-lint	bash -n and ShellCheck on every script
manifest	Required files exist; manifest never lists a secrets file
secret-hygiene	No tracked secrets or keys; CID correct; no *-F0 regression
platform-matrix	All nine distro cases; digest-verifies every staged RPM
bundle	Builds a real bundle and asserts it carries no secrets

Deploy — manual dispatch

Actions → Deploy security tools → Run workflow.

INPUT	VALUE
targets	Comma-separated hosts, or a path to a host file on the runner
components	all or a specific component
mode	plan → verify → deploy
ssh_user	Login on the targets

One-time setup:

```
# Self-hosted runner, labelled: self-hosted, linux, snowcommander
# (GitHub-hosted runners cannot reach *.strykercorp.com)

# Packages on the runner's local disk - not in git
sudo mkdir -p /opt/snowcommander-packages/{crowdstrike,tanium}
sudo cp crowdstrike/*.rpm /opt/snowcommander-packages/crowdstrike/
sudo cp tanium/*.rpm tanium/tanium-init.dat /opt/snowcommander-packages/tanium/
sudo chmod 600 /opt/snowcommander-packages/tanium/tanium-init.dat
```

Repository secrets: DEPLOY_SSH_KEY, SSH_KNOWN_HOSTS, CMDBSYNC_PASSWORD, AZCM_SP_CLIENT_ID, AZCM_SP_SECRET, AZCM_SUBSCRIPTION_ID, AZCM_TENANT_ID, RHSM_PASSWORD.

Repository variables: FALCON_CID, AZCM_RESOURCE_GROUP, AZCM_LOCATION, AZCM_CLOUD, AZCM_TAGS, SYSLOG_SERVER1, SYSLOG_SERVER2, SYSLOG_PORT, CMDBSYNC_UID, CMDBSYNC_GID.

Environment: create production with required reviewers — this gates deploy.

Git CLI

```
# Clone
git clone https://github.com/Kiruthika-syk/Snowcommander-scripts.git
cd Snowcommander-scripts

# Packages are NOT in git (~548 MB, near GitHub's 100 MB per-file limit).
# Stage them from the runner or an internal artifact store:
cp -r /opt/snowcommander-packages/crowdstrike ./crowdstrike
cp -r /opt/snowcommander-packages/tanium ./tanium

# Verify what landed
./lib/pkg_select.sh matrix

# Normal change cycle
git checkout -b fix/whatever
git add -A
git diff --cached --name-only | grep -E 'securitytools\.env$|\.rpm$|tanium-init\.dat$' \
    && echo "STOP - secret or package staged" || echo "clean"
git commit -m "..."
git push -u origin fix/whatever
```

CI runs automatically on push and will fail the build if a secrets file is ever tracked.

Host this site

This page is a single self-contained HTML file with no external scripts, styles, fonts or images, so any static web server can serve it and it works offline. A printable PDF copy lives at `guide.pdf` — regenerate it after editing with `docs/build-guide-pdf.sh`.

Container

```
cd /home/tpx-admin/snowcommander_scripts

podman build -t sectools-docs:latest -f docs/Dockerfile docs/

podman run -d --name sectools-docs \
    -p 8080:8080 \
    --read-only --tmpfs /tmp \
    --cap-drop ALL \
    --security-opt no-new-privileges \
    sectools-docs:latest
```

Then browse `http://<host>:8080/`. Swap `podman` for `docker` if the daemon is running.

Why nginx and not Tomcat

Tomcat is a Java servlet container. There are no servlets, JSPs or Java here — it is one HTML file — so Tomcat would add a JVM and a considerably larger attack surface for no functional gain. The `nginx:alpine` image is a few megabytes and serves static content directly.

Hardening applied

CONTROL	HOW
Unprivileged user	Runs as <code>nginx</code> (uid 101), not root
No privileged port	Listens on 8080, so no <code>NET_BIND_SERVICE</code> capability
Read-only root filesystem	Temp paths redirected to <code>/tmp</code> tmpfs
All capabilities dropped	<code>--cap-drop ALL</code>
Content Security Policy	<code>default-src 'none'</code> — safe because nothing is loaded externally
Version disclosure off	<code>server_tokens off</code>
Health endpoint	<code>/healthz</code> , used by the container <code>HEALTHCHECK</code>

Without a container

```
# Any existing nginx or httpd
sudo cp docs/index.html /usr/share/nginx/html/index.html

# Or a throwaway server for a quick look
cd docs && python3 -m http.server 8080
```

GitHub Pages

Settings → Pages → Source: `main` / `docs`. No build step is needed because the file has no dependencies.

Exit codes

CODE	MEANING	EMITTED BY
0	Success	all
2	Usage error	all
3	Unsupported OS	security.sh
4	Unsupported architecture	security.sh
5	Package selection or validation failed	security.sh
6	Insufficient privilege	security.sh
7	Scope violation — outside SnowCommander/Templates	vsphere_provision.py
10	Partial — some components succeeded	security.sh
11	All components failed	security.sh
20 / 21	Fleet partial / all hosts failed	deploy_targets.sh
3x	Harness stage x failed (35 = stage 5)	e2e_validate.sh

Troubleshooting

SYMPTOM	CAUSE AND FIX
Stage 1 fails, exit 3	Bad credentials, or add <code>--insecure</code> for a self-signed vCenter certificate
Stage 2 fails, exit 4	Template is not inside <code>SnowCommander/Templates</code> . Run <code>--list-templates</code>
exit 7	Scope violation — the object lies outside the permitted folder. Working as intended
Stage 3 IP timeout	No VMware Tools in the template, or the portgroup has no DHCP
Stage 4 SSH failure	Template needs <code>PasswordAuthentication yes</code> , and sudo for the login user
Stage 5 syslog fails	RHSM registration failed — check <code>RHSM_USERNAME</code> / <code>RHSM_PASSWORD</code>
<code>STAGE_FAILED</code> in fan-out	Unsupported OS/arch, or no package pair — e.g. el7 aarch64 Falcon
Falcon active but no AID	Sensor is running but has not reached the cloud. Check egress to <code>ts01-b.cloudsink.net:443</code>
<code>UNREACHABLE</code>	SSH failed. Test manually with the same key or password
<code>NO_SUDO</code>	Passwordless sudo missing on the target
Corrupt package	<code>rpm -K</code> failed. Re-stage from the vendor; run <code>./lib/pkg_select.sh matrix</code>

Security

CONTROL	IMPLEMENTATION
vSphere scope	Hard-locked to <code>SnowCommander/Templates</code> ; checked again before any mutating call
Non-destructive default	Clone preserves templates. <code>--mode convert</code> requires an explicit acknowledgement flag
No secrets in images	Build fails if a secrets file is in the context
No secrets in git	<code>.gitignore</code> plus a CI job that fails on a tracked secrets file
No secrets in <code>ps</code>	Everything via environment or stdin, never command arguments
Secret redaction	<code>sed</code> filter masks values in stdout and logs, including third-party output
Secrets removed after use	<code>securitytools.env</code> shredded from the target at stage 8
Non-root container	<code>uid=998(deployer)</code> , <code>cap_drop: ALL</code> , read-only root filesystem
Package integrity	<code>rpm -K</code> digest verification at build, at stage time and on target
Config safety	Existing files backed up before any overwrite
Gated deploys	GitHub <code>production</code> environment with required reviewers

Outstanding. Credentials that previously sat in a world-readable `securitytools.env` — the Azure service-principal secret, the cmdbsync password and the RHSM password — should be rotated. The file is now mode 600 and gitignored, but anything previously exposed should be considered compromised.